CLEARCUT

Invoicing that agents can call, and people can trust

YEAR	ROLE	TEAM	STATUS
2026	Product, Rust engineering, protocol design	Solo build	Live · alpha

Summary

An agent-native invoicing core for South Africa: one Rust codebase exposed as a REST API, an MCP server and a Claude plugin, with the VAT rules already written as tested code.

The problem

An agent can draft an invoice in seconds. Getting the money right, and making the result defensible to an accountant, is the hard part. South African tax invoices have thresholds, VAT has rounding rules, and an agent that invents a figure is worse than no agent at all.

ClearCut puts the determinism first. The core is a Rust library that knows how to calculate, validate and explain an invoice. Agents are a surface on top of it, not the source of truth.

What is built

- Correct money by construction: integer cents, deterministic rounding, 15% VAT and the South African tax-invoice thresholds (full above R5 000, abridged down to R50, none at or below) as tested code.
- An MCP server over stdio with three tools (calculate_invoice, za_required_tax_invoice, check_status_transition), a Claude plugin manifest and a typed TypeScript SDK with an upload client.
- A multi-tenant REST API on PostgreSQL: row-level security that fails closed, a restricted service role, hashed and scoped API keys, audited key management and a streamed document-upload endpoint.
- A deterministic core ready for models: extraction validation, cited event derivation and template narration behind a number guard that rejects any figure the evidence does not contain.

Decisions that matter

Integer cents

Choice. Money is stored and calculated as integer cents with explicit rounding.

Why. Totals do not drift between services.

Trade-off. Every boundary needs a conversion rule.

Tax rules as tested code

Choice. VAT and invoice thresholds live in the library with tests beside them.

Why. An agent cannot invent a figure the rules do not allow.

Trade-off. The rules must be maintained as tax law changes.

One core, three surfaces

Choice. REST, MCP and a Claude plugin all call the same Rust core.

Why. Agent access does not fork the source of truth.

Trade-off. Each surface needs its own adapter and tests.

Stack

`Rust` `axum` `PostgreSQL` `MCP` `TypeScript` `Claude plugin`

Where to look

- Open live site: https://145-241-100-85.sslip.io/
- Source on GitHub: https://github.com/konethegreat/clearcut
- Case study: konethegreat.github.io/work/clearcut